Meeting 8 – Part 2

Memory Architecture (Final Design)

Project: Digital Harmonium Workstation
Version: Final Discussion Draft
Status: Approved Design Philosophy

8.1 Objective

This meeting establishes the complete memory architecture of the Digital Harmonium Workstation.

Unlike traditional software design, embedded systems possess a fixed amount of physical memory. Every byte must therefore have a clearly defined purpose before implementation begins.

The objective of this document is to answer a single engineering question:

> **Where does every byte of memory belong during runtime?**

By defining this architecture before implementation, the software becomes:

- Deterministic
- Predictable
- Easier to debug
- Easier to optimize
- Easier to extend

This document forms the foundation of all future software development.

8.2 Available Memory Resources

The ESP32 provides three major storage resources.

---

1. Internal SRAM

Total Available:

520 KB

This memory is used only while the system is powered.

It stores:

- Runtime variables
- Objects
- Voice structures
- Audio buffers
- Task stacks
- FreeRTOS data
- DMA buffers
- LCD state
- Recorder buffers

SRAM provides the fastest access in the system and is reserved exclusively for active runtime information.

---

2. Internal Flash Memory

Total Available:

4 MB

Flash memory is non-volatile.

Unlike SRAM, it retains data after power loss.

Flash stores:

- Firmware
- Operating software
- Current instrument package (VHL)
- Configuration
- Factory data

Unlike SRAM, Flash is considerably larger but slower to write.

Reading from Flash is sufficiently fast for normal execution and instrument playback.

---

3. SD Card Storage

Expandable external storage.

The SD card is **not** considered runtime memory.

Its purpose is permanent storage.

Examples:

- Instrument Library
- Additional VHL packages
- Recordings
- Presets
- User backups
- Future expansion

The SD card functions as an archive rather than active memory.

8.3 Fundamental Memory Philosophy

The workstation follows a strict memory philosophy.

RAM is for things that are alive.

Examples:

- Pressed keys
- Active voices
- Audio buffers
- Running tasks
- LCD data

These objects change continuously.

Therefore they belong in SRAM.

Flash is for things that are currently installed.

Examples:

- Firmware
- Loaded Harmonium VHL
- Loaded Organ VHL
- Configuration

Flash behaves like the installed operating environment.

SD Card is for things that are sleeping.

Examples:

- Entire instrument library
- Saved recordings
- Additional sound banks
- User files

The SD card serves as long-term storage.

8.4 Instrument Loading Philosophy

A major architectural decision concerns instrument storage.

Earlier concepts considered streaming every instrument directly from the SD card.

After discussion, a cleaner architecture was selected.

Final Design

```
Instrument Library
        │
        │
     SD Card
        │
        ▼
 Selected Instrument
        │
        ▼
 Internal Flash
        │
        ▼
 Runtime Playback
```

Meaning:

The SD card stores every available instrument.

When the user selects an instrument:

- The selected VHL package is copied into Flash.

- Runtime playback accesses Flash directly.
- The SD card is no longer involved during normal performance.

Advantages:

- Lower latency
- No SD card stalls
- Cleaner runtime architecture
- Smaller RAM usage
- Unlimited instrument library

8.5 SRAM Allocation Strategy

The entire SRAM is divided into dedicated regions.

Each region has a single responsibility.

No region should interfere with another.

Region 1 — FreeRTOS Kernel

Reserved:

32 KB

Purpose:

- Scheduler
- Queues
- Semaphores
- System objects
- Idle task
- Timer task

This memory remains allocated throughout system operation.

Region 2 — Task Stacks

Reserved:

32 KB

Purpose:

Dedicated execution stacks for all software tasks.

Including:

- Keyboard Task
- Display Task
- Recorder Task
- Storage Task
- Background Task
- Loader Task
- USB Task
- Settings Task

Each task owns an independent stack.

Region 3 — DMA Buffers

Reserved:

8 KB

Purpose:

Double-buffered audio output.

Configuration:

- Buffer A
- Buffer B

The CPU renders audio into one buffer while the I^2S peripheral transmits the other.

These buffers are timing-critical and must never be modified by unrelated modules.

Region 4 — Audio Mixer Workspace

Reserved:

8 KB

Purpose:

Temporary floating-point accumulation during audio mixing.

Stores:

- Mixed voice output
- Intermediate sample calculations
- Limiter input
- Master gain processing

Destroyed and regenerated every audio block.

Region 5 — Voice Manager Objects

Reserved:

16 KB

Purpose:

Stores every currently active voice.

Each voice object contains:

- Voice ID
- Key number
- Velocity
- Phase accumulator
- Envelope state
- Playback position
- Loop information
- Gain
- Pitch ratio
- Runtime flags

Only active voices occupy this region.

Region 6 — Instrument Runtime Metadata

Reserved:

32 KB

Purpose:

Stores information describing the currently loaded instrument.

Examples:

- Sample lookup tables
- Key mappings
- Region tables
- Velocity regions
- Root key information
- Instrument parameters
- Runtime pointers

Raw sample data is **not** duplicated into SRAM.

Region 7 — Recorder Buffer

Reserved:

32 KB

Purpose:

Temporary recording queue.

Recording never writes directly to the SD card.

Instead:

```
Key Events

↓

Recorder Buffer

↓

Background Storage Task

↓

SD Card
```

This prevents audio interruptions.

Region 8 — Display Manager

Reserved:

4 KB

Purpose:

Stores:

- LCD text
- Current screen
- Menu state
- Cursor position
- Status indicators

Small but permanently allocated.

Region 9 — Loader Workspace

Reserved:

32 KB

Purpose:

Temporary workspace while:

- Loading instruments
- Reading SD files
- Parsing VHL packages
- Copying Flash data

Freed immediately after loading completes.

Region 10 — DSP Scratch Memory

Reserved:

32 KB

Purpose:

Temporary mathematical workspace.

Examples:

- Interpolation
- Envelope calculations
- Reverb processing
- Chorus processing
- Future DSP algorithms

Exists purely to simplify runtime processing.

Region 11 — General Runtime Heap

Reserved:

Approximately **250 KB**

Purpose:

General dynamic allocation.

This region intentionally remains largely unused during normal operation.

It provides:

- Safety margin
- Future expansion
- Library allocations
- Unexpected runtime requirements

The workstation should never attempt to consume the entire available heap.

Maintaining significant free memory improves long-term system stability.

8.6 Flash Memory Architecture Philosophy

Internal Flash serves as the workstation's installed environment.

Unlike SRAM, its contents persist after power loss.

Flash stores:

- Firmware
- Runtime software
- Currently installed instrument package
- Persistent settings
- Calibration information

Whenever a new instrument is selected, the Instrument Loader replaces the previous VHL package stored in the dedicated Flash partition.

Only the active instrument occupies Flash at any given time.

This minimizes runtime complexity while allowing the SD card to store an unlimited library of additional instruments.

8.7 SD Card Philosophy

The SD card is treated purely as external storage.

Its responsibilities include:

- Instrument library
- User recordings
- Preset backups
- Future expansion
- Firmware updates (if implemented)

Normal instrument playback should not depend on continuous SD card access after an instrument has been loaded into Flash.

8.8 Memory Safety Principles

The architecture follows several safety rules.

1. Every subsystem owns a dedicated memory region.

2. DMA buffers remain isolated from application logic.

3. Runtime heap is never intentionally exhausted.

4. Large temporary operations occur inside Loader Workspace.

5. Audio processing never allocates memory dynamically during playback.

6. Recording is buffered before storage.

7. Instrument metadata resides in SRAM while instrument content resides in Flash.

8.9 Final Memory Hierarchy

```
ESP32

├── Internal SRAM (520 KB)
│   ├── FreeRTOS
│   ├── Task Stacks
│   ├── DMA Buffers
│   ├── Audio Mixer
│   ├── Voice Objects
│   ├── Instrument Metadata
│   ├── Recorder Buffer
│   ├── Display Manager
│   ├── Loader Workspace
│   ├── DSP Scratch
│   └── General Heap
│
├── Internal Flash (4 MB)
│   ├── Firmware
│   ├── Runtime Software
│   ├── Active Instrument VHL
│   └── Persistent Settings
│
└── SD Card
    ├── Instrument Library
    ├── Recordings
    ├── Presets
    ├── Backups
    └── Future Expansion
```

8.10 Meeting Summary

This meeting finalized the memory organization of the Digital Harmonium Workstation.

The architecture establishes a clear separation between volatile runtime memory (SRAM), installed runtime resources (Flash), and permanent archival storage (SD Card).

Every major subsystem now possesses a dedicated memory allocation strategy, ensuring deterministic behavior, efficient resource usage, and a scalable foundation for future development.

This memory architecture will remain one of the core reference documents for all subsequent software implementation phases.

End of Meeting 8 – Part 2: Memory Architecture